DEMANDPREDICT

DATA SCIENCE PROJECT

OCTOBER 2026

PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING OF THE ANNA UNIVERSITY

PROJECT WORK

BATCH 2024 - 2028

Submitted by

_____

_____

Under the Guidance of

_____

Associate Professor, CSE

Sri Eshwar College of Engineering

(An Autonomous Institution)

Kinathukadavu (Tk), Coimbatore - 641 202, Tamil Nadu

Approved by AICTE, New Delhi and Affiliated to Anna University, Chennai

DEMANDPREDICT

RETAIL DEMAND FORECASTING AND CUSTOMER ANALYTICS

DATA SCIENCE PROJECT REPORT

Submitted by

_____

_____

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

SRI ESHWAR COLLEGE OF ENGINEERING

(AN AUTONOMOUS INSTITUTION)

COIMBATORE - 641 202

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"DemandPredict"** was carried out by

_____

_____

who carried out the Data Science Project work under my supervision.

_____

SIGNATURE

Head of the Department

Department of Computer
Science and Engineering,

Sri Eshwar College of
Engineering,
Coimbatore - 641 202.

_____

SIGNATURE

Project Guide

_____

Department of Computer
Science and Engineering,

Sri Eshwar College of
Engineering,
Coimbatore - 641 202.

Submitted for the End Semester Project examination **U23AD491 Data Science** Project viva-voce held on .....

.....

(Internal Examiner)

.....

(External Examiner)

DECLARATION

I declare that the project entitled **"DemandPredict"**, submitted in partial fulfilment of the requirements for the Bachelor of Engineering degree in Computer Science and Engineering, is a record of the project work carried out under the supervision and guidance of the project guide, Department of Computer Science and Engineering, Sri Eshwar College of Engineering, Coimbatore.

The report describes the implemented application, its methods and the results obtained from the project demonstration dataset. Sources used in the study are acknowledged in the references. The work has not been submitted by me for the award of another degree or diploma.

Place: Coimbatore

Date: .....

Signature of the Student

ACKNOWLEDGEMENT

I express my sincere gratitude to the management of Sri Eshwar College of Engineering for providing the academic environment and facilities required to carry out this Data Science project.

I thank the Principal and the Head of the Department of Computer Science and Engineering for their encouragement and support during the development of the project.

I extend my gratitude to my project guide for guidance, constructive feedback and assistance in shaping the project methodology and presentation.

I also thank the faculty members of the Department of Computer Science and Engineering for sharing their knowledge and supporting the learning process. Their teaching helped establish the foundations in programming, databases, statistics and machine learning used in this project.

I acknowledge the developers and maintainers of the open-source libraries used in DemandPredict. Their work makes it possible to combine an interactive application with reproducible data analysis and machine learning.

Finally, I thank my family and friends for their encouragement and support throughout the completion of this project.

Chapter	Table of Contents	Page No.
	ABSTRACT	i
	LIST OF FIGURES	ii
1	**INTRODUCTION**	1
2	**SYSTEM ANALYSIS AND DESIGN**	2
	2.1 Literature Survey	
	2.2 Existing Product	
	2.3 Problem Statement	
3	**PROPOSED SYSTEM**	3
	3.1 Overview	
	3.2 Block Diagram	
4	**PROJECT DESCRIPTION**	4
	4.1 Dataset Collection and Description	4
	4.2 Methodology	5-7
	4.3 Implementation	8-9
5	**RESULTS AND INFERENCES**	10
	5.1 Model Evaluation	10
	5.2 Application Outputs	11-16
	5.3 Testing and Verification	17
6	**CONCLUSION AND FUTURE SCOPE**	18
	6.1 Conclusion	
	6.2 Future Scope	
	REFERENCES	19

ABSTRACT

DemandPredict is a retail analytics application that combines demand forecasting, customer purchase propensity, behavioural segmentation and inventory planning in a single dashboard. Retail records often remain distributed across transaction exports, customer lists and product catalogues, making it difficult to connect demand patterns with customer behaviour and available stock. This project provides a structured workflow for importing such records and producing interpretable analytical outputs.

The system uses React and TypeScript for the interface, FastAPI for the application programming interface, and SQLite with SQLAlchemy for persistent storage. Five related CSV datasets represent customers, products, promotions, behavioural events and orders. Schema validation, reference checks and transactional imports maintain consistency before analysis begins.

Demand is forecast using a moving-average baseline, Prophet, XGBoost, LightGBM and a validation-weighted ensemble. Chronological holdouts separate model evaluation from fitting. Regularized logistic regression estimates customer-product purchase probability from historical behaviour, while linear SHAP contributions describe the fitted model's feature effects. RFM features and K-means support customer segmentation. Inventory recommendations combine recent demand, stock and a planning buffer.

The implemented system was evaluated using a synthetic demonstration dataset with 120 customers, 24 products, 1,481 order rows and 12,039 events. The purchase model achieved a ROC AUC of 0.7265 and a Brier score of 0.05744 on a later 30-day window. Forecast results varied across categories, demonstrating the need to compare alternatives against a simple baseline. Application verification covered API behaviour, import integrity and browser workflows. The project demonstrates an integrated, reproducible local decision-support workflow; its synthetic-data results do not establish production effectiveness.

Keywords: demand forecasting, purchase propensity, retail analytics, RFM, K-means, explainable machine learning.

LIST OF FIGURES

Figure	Description	Page
3.1	System block diagram	3
4.1	Methodology and chronological validation flow	5
5.1	Retail overview dashboard	11
5.2	Demand forecasting and model validation	12
5.3	Customer recommendations and explanation	13
5.4	Dataset import and status interface	14
5.5	Customer segmentation interface	15
5.6	Inventory planning interface	16

LIST OF TABLES

Table	Description	Page
4.1	Demonstration dataset and key attributes	4
4.2	Application technologies and responsibilities	8
5.1	Category forecast MAE on the holdout	10
5.2	Purchase propensity validation	10
5.3	Verification summary	17

1. INTRODUCTION

Retail businesses record purchases, product views, shopping-cart additions and stock levels through their digital systems. These records can support decisions about replenishment and customer engagement when they are organized into a consistent analytical workflow. Sales summaries alone describe what happened; useful planning also requires estimates of future demand and an understanding of the customers behind those transactions.

DemandPredict addresses this need through a local web application for retail demand forecasting and customer analytics. It brings validated data import, historical summaries, forecasting, customer recommendations, segmentation and inventory suggestions into one interface. Users can move from an overview of the business to a specific product or customer without preparing separate analysis scripts for each task.

The project has four main analytical objectives. The first is to compare daily demand forecasts from a baseline and several machine learning models. The second is to estimate the probability that a customer will purchase a product within a chosen horizon. The third is to identify behavioural customer groups using recency, frequency and monetary value. The fourth is to translate recent demand and current stock into understandable inventory planning indicators.

A reliable workflow must also handle invalid inputs and communicate uncertainty. The application therefore checks file schemas, data types and related record identifiers before updating the database. Forecasts display the final observed date, and evaluation uses later time periods that were excluded from fitting. Purchase explanations are presented as model associations rather than causal reasons for customer behaviour.

The demonstration uses generated retail records so that the complete workflow can be reproduced without relying on private customer information. The generated history contains variation in product preferences, purchasing activity and calendar patterns. Evaluation on this dataset verifies the implemented analytical process, while evaluation on real retail data remains necessary before operational adoption.

The scope is a trusted, single-user project environment. DemandPredict provides estimates and recommendations for human review; it does not place purchase orders or execute marketing campaigns. Subsequent chapters describe the system analysis, proposed architecture, dataset, methodology, implementation and measured results.

2. SYSTEM ANALYSIS AND DESIGN

2.1 Literature Survey

Taylor and Letham describe forecasting at scale using an interpretable model with trend and seasonal components [1]. This motivates the inclusion of Prophet as one candidate for retail daily demand. In this project, weekly seasonality is enabled; annual seasonality requires at least a year of observations.

Chen and Guestrin introduce XGBoost, a scalable tree boosting system [2]. Ke et al. present LightGBM and techniques for efficient gradient boosting [3]. DemandPredict uses their regression implementations with lag, rolling and calendar features. The LightGBM configuration uses its standard regressor; the project does not claim a custom implementation of the paper's algorithms.

Lundberg and Lee develop SHAP, a framework for additive feature attributions [4]. DemandPredict uses the closed-form linear, independent-feature interpretation for logistic regression, explaining 30-day log odds relative to mean training features. It does not use a general-purpose SHAP package or claim causal explanations.

2.2 Existing Product

The existing workflow considered here consists of spreadsheets, sales reports and separate modelling notebooks. These can summarize transactions or train individual models, but require manual coordination to maintain consistent dates, customer identifiers and product definitions. A forecast prepared separately from stock and customer records also requires additional interpretation before it can support a planning decision.

DemandPredict consolidates those steps into a purpose-built application. This comparison describes the workflow addressed by the project; it is not a benchmark of commercial retail products.

2.3 Problem Statement

Develop a reproducible retail analytics system that accepts related CSV datasets, validates their integrity and provides demand forecasts, customer purchase estimates, behavioural segments and inventory recommendations. The system must report validation results, explain key outputs and handle insufficient history without presenting unsupported certainty.

The design prioritizes data consistency, chronological evaluation, a usable dashboard and traceable outputs. Production-scale processing, authenticated multi-user access and automatic replenishment are outside the implemented scope.

3. PROPOSED SYSTEM

3.1 Overview

DemandPredict uses a three-layer architecture. The presentation layer provides six dashboard screens. The FastAPI application validates requests and coordinates analytical services. SQLite stores the related retail records, while Python modules generate forecasts and customer estimates from the same imported data.

A user imports the five supported CSV tables or loads the fictional demonstration dataset into an empty database. The application checks all supplied files before committing changes. Analytics then operate on a common dataset snapshot. Forecasts, customer recommendations and inventory plans can be exported as CSV for further review.

3.2 Block Diagram

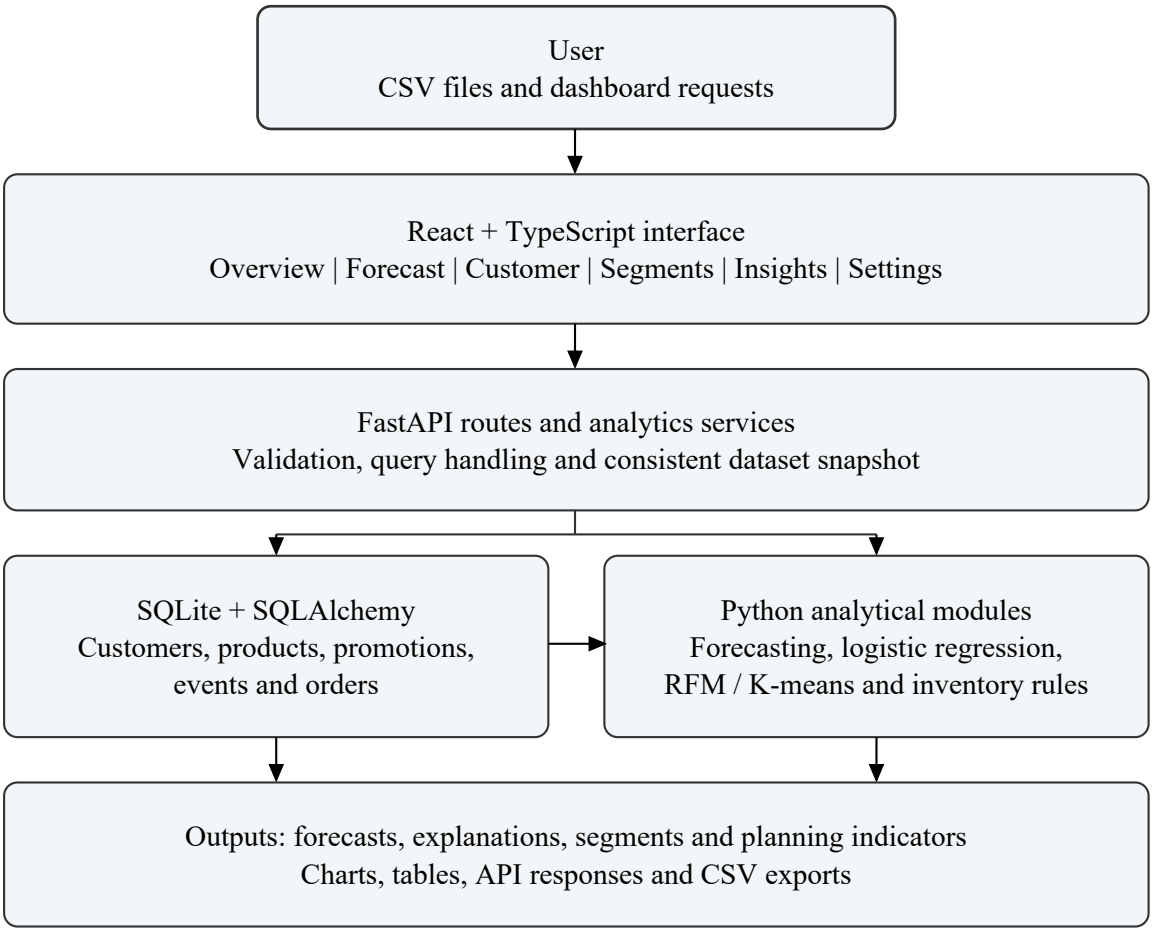


Figure 3.1 System block diagram

The analytical modules read validated records and return derived results. Imported changes invalidate the cached customer model through a data fingerprint, so subsequent outputs reflect the updated dataset.

4. PROJECT DESCRIPTION

4.1 Dataset Collection and Description

The system accepts user-provided CSV files and includes a seeded synthetic generator for reproducible demonstrations. The evaluated database contains 180 days of order history, from 22 April to 18 October 2026. Its customers and retail activities are fictional. Values in the interface are interpreted as Indian rupees (INR).

Table 4.1 Demonstration dataset and key attributes

Table	Rows	Representative attributes
Customers	120	Customer ID, name, email, signup date, location
Products	24	Product ID, category, price, launch date, stock and reorder point
Promotions	1	Promotion ID, type, active dates and discount details
Orders	1,481	Customer ID, product ID, date, quantity, price, status and session
Events	12,039	Customer ID, product ID, timestamp, event type and session

Customers and products each have one-to-many relationships with orders and events. Optional promotion references connect eligible transaction or behaviour records to promotions. Primary identifiers are preserved as strings. Foreign-key checks reject references to records that do not exist after an import.

The 24 products span eight categories: Beauty and Personal Care, Books and Media, Clothing, Electronics, Grocery and Gourmet, Home and Kitchen, Sports and Fitness, and Toys and Games. Product preferences and time-dependent purchasing patterns are generated with seed 42. These patterns are demonstration assumptions, not observations from a commercial retailer.

Sales calculations include confirmed, shipped and delivered orders. Pending, cancelled and returned orders are excluded. An order row is a stored transaction record; it is not necessarily a unique purchase session. Frequency for customer segmentation is calculated from distinct eligible sessions.

Missing calendar days within the observed series are filled with zero demand. Product history begins no earlier than its launch date. The application limits daily forecasting history to the latest 730 days and anchors forecasts after the dataset's latest order or event date.

4.2 Methodology

The methodology separates data ingestion, feature construction, temporal evaluation and presentation. The same data definitions are used for dashboard summaries and model inputs so that outputs can be interpreted against the recorded history.

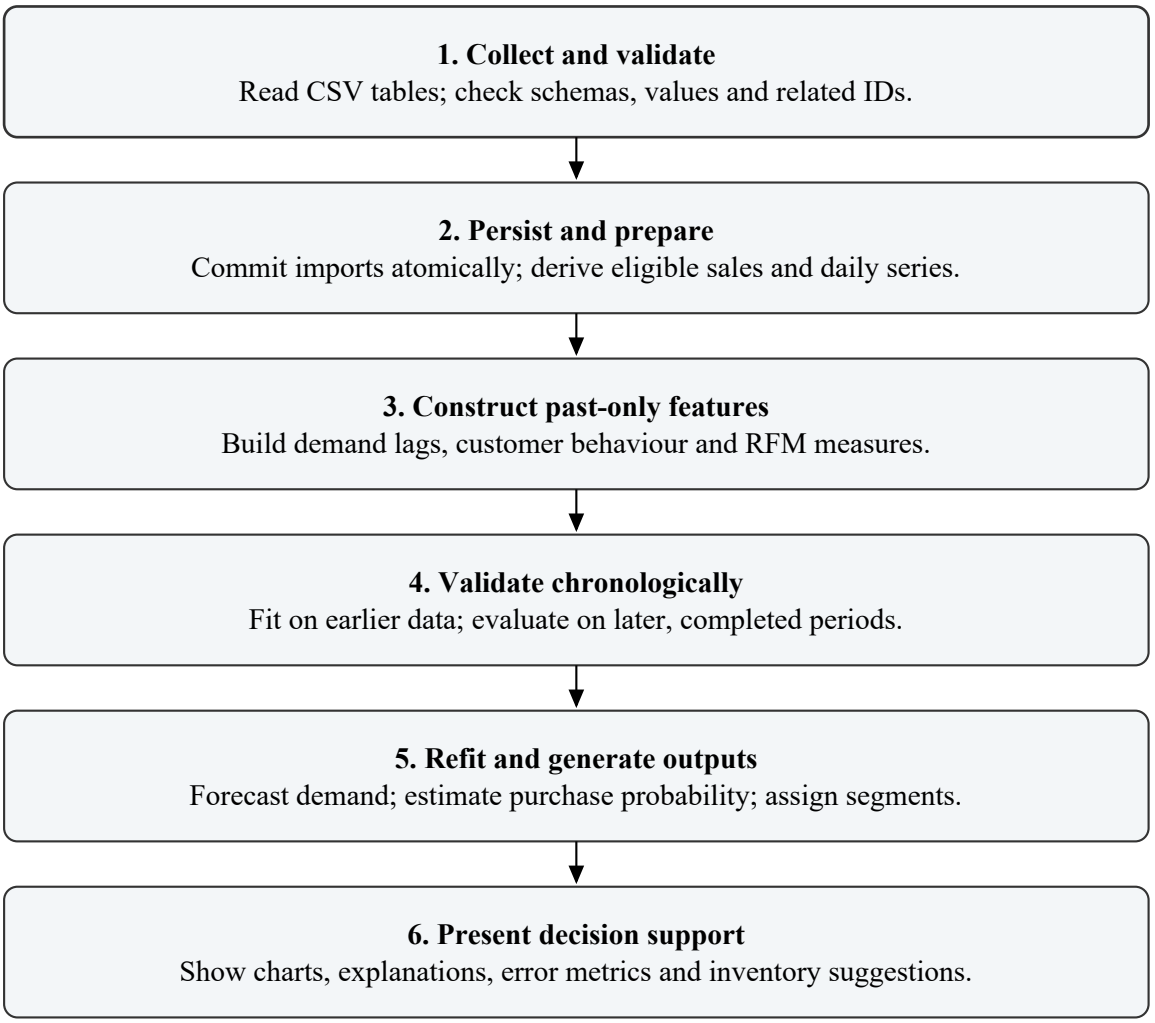


Figure 4.1 Methodology and chronological validation flow

Evaluation discipline

Demand forecast metrics use the final 14 observed days when sufficient history is available. Ensemble weights use an earlier, separate window, preventing their selection from using the final evaluation targets. The final forecast is refitted on all observations after validation.

Purchase propensity uses an earlier 30-day outcome window for fitting and a later 30-day window for validation. Features at each cutoff use only prior records. Final refitting combines the two completed windows. This avoids using future customer activity in the features.

4.2 Methodology (continued)

Demand forecasting

The moving-average baseline predicts the mean of the last 28 daily observations. Prophet models trend and weekly seasonality [1]. XGBoost and LightGBM fit boosted regression trees [2, 3]. Their inputs include demand at lags 1, 7 and 14; means over 7, 14 and 28 days; seven-day standard deviation; and day-of-week, month and day-of-year.

The tree models use 60 estimators, maximum depth 3, learning rate 0.06 and random seed 42. LightGBM uses seven leaves and a minimum of five observations per child. Future tree predictions are recursive: each predicted value is appended to the history before the next day is estimated. Outputs are constrained to non-negative demand.

The ensemble combines the four model forecasts. For sufficiently long histories, inverse-MAE weights are estimated from the 14 days preceding the final validation window. Shorter eligible histories use equal weights. Fewer than 49 observations or constant demand trigger a moving-average fallback. Fewer than 14 observations do not produce validation metrics.

Error measures and forecast ranges

Let y(t) be observed demand, f(t) the prediction and n the number of validation days. The implemented error measures are:

$$MAE = \text{sum } |y(t) - f(t)| / n$$

$$RMSE = \text{sqrt}(\text{sum } (y(t) - f(t))^2 / n)$$

$$MAPE = 100 \times \text{mean}(|y(t) - f(t)| / y(t)), \text{ for } y(t) > 0$$

$$WMAPE = 100 \times \text{sum } |y(t) - f(t)| / \text{sum } y(t)$$

MAPE excludes zero-demand days. WMAPE is omitted when the total observed holdout demand is zero. These rules prevent division by zero, but MAPE can still underrepresent the difficulty of intermittent demand.

Forecast bands use validation residual error with an approximate 1.96 x RMSE multiplier and increasing horizon width. Lower bounds are truncated at zero. These bands are heuristic ranges; their nominal probability coverage has not been calibrated. They should support interpretation rather than be treated as guaranteed stock requirements.

4.2 Methodology (continued)

Customer purchase propensity and explanations

A regularized logistic regression model uses eight customer-product features: prior product purchases, recent views, recent cart additions, customer order count, product order count, category affinity, days since the product was purchased, and relative price. The pipeline applies feature transformations and standardization before fitting. Uniform pair sampling preserves the observed label prevalence within each temporal window.

The target is whether a customer purchases the product in the next 30 days. For a horizon h, the application converts the 30-day probability p30 using $p(h) = 1 - (1 - p30)^{h/30}$. This assumes a constant purchase hazard. Expected quantity multiplies the probability by historical conditional purchase quantity, and expected revenue also multiplies by product price.

For the linear model, a feature contribution is its standardized deviation from the training mean multiplied by its coefficient. The largest absolute contributions explain 30-day log odds under an independent-feature assumption [4]. Correlated features can complicate interpretation. Insufficient history or a single target class produces an explicitly labelled smoothed-history estimate.

Behavioural customer segmentation

RFM features represent recency in days, purchase frequency in distinct sessions, and eligible monetary spend. Purchasers are clustered using standardized log(1 + RFM) values and K-means, with up to four distinct clusters, random seed 42 and ten initializations. Labels describe the resulting cluster profiles rather than using the synthetic generator's latent segment labels.

Customers without eligible purchases form a separate group. Purchasers inactive for at least 60 days are labelled at risk. Discount-heavy occasional buyers can receive a promotion-responsive label. These rules aid interpretation but do not represent a validated churn model.

Inventory planning

Expected demand is the last 28 days' mean daily units multiplied by the planning horizon. Stockout risk compares that estimate and a 20% buffer with available stock. Suggested replenishment is max(0, ceil(1.2 x demand + reorder point - stock)) when expected demand is positive. More than 90 days of stock cover indicates overstock. Lead times, inbound orders and demand hidden by past stockouts are not included.

4.3 Implementation

Table 4.2 Application technologies and responsibilities

Component	Technology and purpose
Presentation	React 18, TypeScript, Vite, Tailwind CSS and Recharts; interactive views, charts, routing and themes.
API	FastAPI, Pydantic and Uvicorn; request validation, API responses and generated API documentation.
Persistence	SQLAlchemy and SQLite; related customer, product, promotion, event and order records.
Data and models	pandas, NumPy, scikit-learn, Prophet, XGBoost and LightGBM; preparation, estimation and evaluation.
Verification	pytest, HTTPX, Playwright, TypeScript, ESLint and Ruff; application and code checks.

Import and API implementation

The upload service rejects missing or unknown columns, invalid dates, non-finite monetary values, invalid quantities and unresolved references. Customer emails must be valid and unique. Individual uploads default to a 100 MB limit. Repeated identifiers are upserted, and all files in one import either succeed together or are rolled back. List-like values use safe literal parsing rather than expression evaluation.

API routes expose product and category forecasts, customer predictions, segments, stock insights, dataset status and CSV templates. Forecast requests restrict the horizon to 1-90 days and model selection to supported identifiers. SQLAlchemy sessions provide access to the same database for all services.

Interface and local operation

The six screens provide responsive navigation, loading and error feedback, empty states and light/dark themes. CSV downloads escape spreadsheet formula prefixes. The local launcher starts the interface on port 5173 and the API on port 8000, both on localhost. Documentation is available through /docs.

Model training and analytics run inside the API process. A fingerprint-based cache avoids reusing a customer model after data changes. The application is intended for a trusted single-user environment and does not implement login or multi-tenant authorization.

4.3 Implementation (continued)

Representative source code

The following excerpts are taken from the implemented project. The first constructs the lag, rolling and calendar predictors used by the tree forecasting models.

```
def features(history, date):
    values = np.asarray(history, dtype=float)
    return [
        values[-1],
        values[-7],
        values[-14],
        values[-7:].mean(),
        values[-14:].mean(),
        values[-28:].mean(),
        values[-7:].std(),
        date.dayofweek,
        date.month,
        date.dayofyear,
    ]
```

Source: ml/forecasting.py

The product forecast route validates the planning horizon and the requested model before delegating to the shared analytical dataset service.

```
@router.get("/forecast/product/{product_id}")
def forecast_product(
    product_id: str,
    horizon: int = Query(30, ge=1, le=90),
    model: str = Query("ensemble", pattern="^(ma|prophet|xgboost|lightgbm|ensemble)$"),
    db: Session = Depends(get_db),
):
    return Dataset(db).forecast(horizon, model, product_id=product_id)
```

Source: backend/app/api/forecast.py

Reproducible execution

Install the Python requirements into the project virtual environment, install the frontend dependencies, and run `npm run dev` from the project root. The Settings screen provides schema examples and the demo-data action. The training pipeline writes the evaluation JSON and customer-model artifact into the models directory. The README records the complete setup and verification commands.

5. RESULTS AND INFERENCES

5.1 Model Evaluation

The saved evaluation uses the dataset through 18 October 2026. Table 5.1 reports daily-unit MAE over the latest 14 days, rounded to three decimals. Lower values indicate smaller absolute errors.

Table 5.1 Category forecast MAE on the holdout

Category	MA	Prophet	XGB	LGBM	Ensemble
Beauty & Personal Care	1.714	1.486	1.924	1.892	1.643
Books & Media	1.980	2.040	1.955	1.868	1.941
Clothing	1.663	1.659	1.813	1.689	1.701
Electronics	1.638	1.614	1.936	1.762	1.659
Grocery & Gourmet	1.490	1.652	1.623	1.517	1.523
Home & Kitchen	1.985	1.881	2.152	1.837	1.952
Sports & Fitness	1.888	1.904	1.769	1.938	1.822
Toys & Games	1.867	1.843	1.887	1.863	1.825
Category mean	**1.778**	**1.760**	**1.882**	**1.796**	**1.758**

MA denotes moving average; XGB and LGBM denote XGBoost and LightGBM. The mean is an unweighted average across categories, not a pooled sales-weighted metric. The ensemble has the lowest category MAE for Toys and Games, while other categories favour individual models. Sparse daily demand also produces WMAPE above 100% for some models and categories. These results do not justify claiming universal superiority.

Table 5.2 Purchase propensity validation

Measure	Observed value
Later-window examples / positive rate	2,880 / 6.81%
ROC AUC	0.7265
Brier score / constant baseline	0.05744 / 0.06364
Examples used in final refit	5,760

The lower Brier score than the constant baseline indicates improved squared-probability error on this window. ROC AUC measures ranking, not percentage accuracy. Neither result establishes calibration or commercial benefit on real customers. Values are sourced from models/evaluation.json.

5.2 Application Outputs

Retail overview

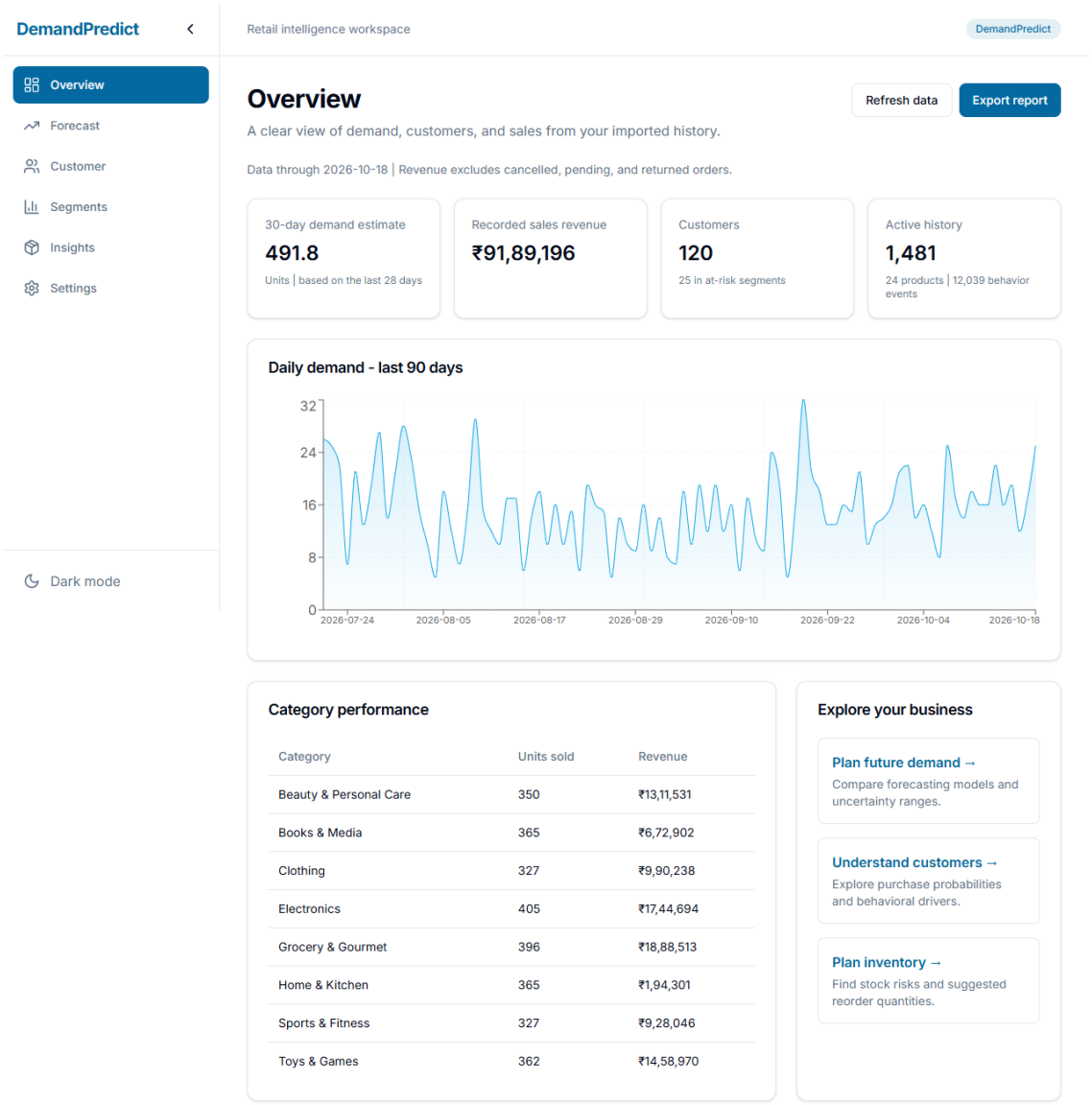


Figure 5.1 Retail overview dashboard

The Overview screen connects the imported data to summary indicators, daily demand and category performance. The captured dataset contains 120 customers, 24 products, 1,481 order rows and 12,039 behaviour events. The 30-day demand estimate is based on the recent daily average, while recorded revenue summarizes eligible historical sales.

The displayed as-of date identifies the end of the dataset. Historical revenue and estimated future demand refer to different periods and should not be compared as if they were the same measure.

5.2 Application Outputs (continued)

Demand forecast

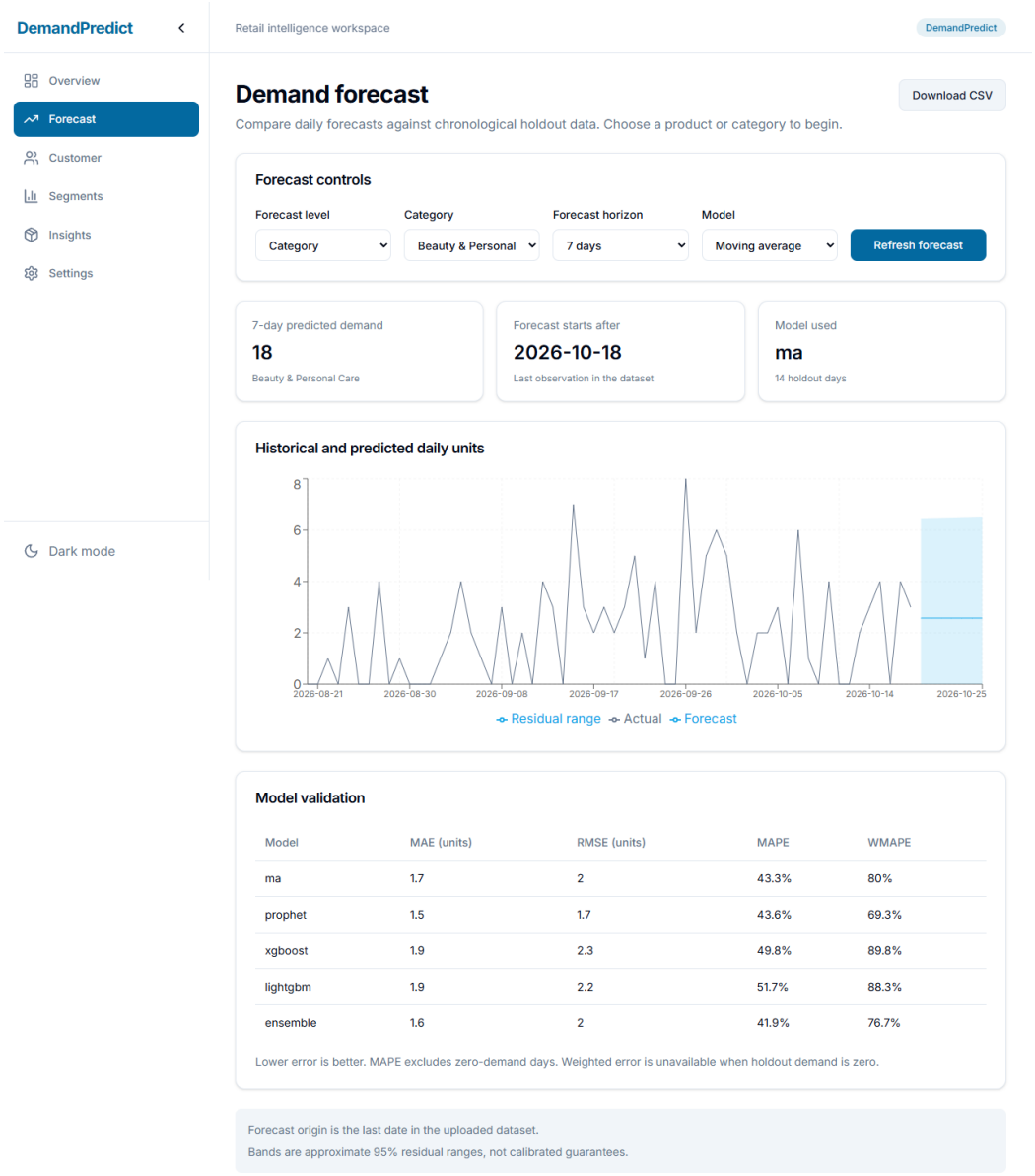


Figure 5.2 Demand forecasting and model validation

The Forecast screen supports product and category selection, a 1-90 day horizon and the five model options. This screenshot shows a seven-day moving-average category forecast. Historical demand, predicted values and approximate residual ranges are presented together, with held-out error measures below the chart.

The user can compare model validation metrics and download the forecast as CSV. Predictions begin after the last observed dataset date; the interface does not present an old dataset as current demand.

5.2 Application Outputs (continued)

Customer analysis

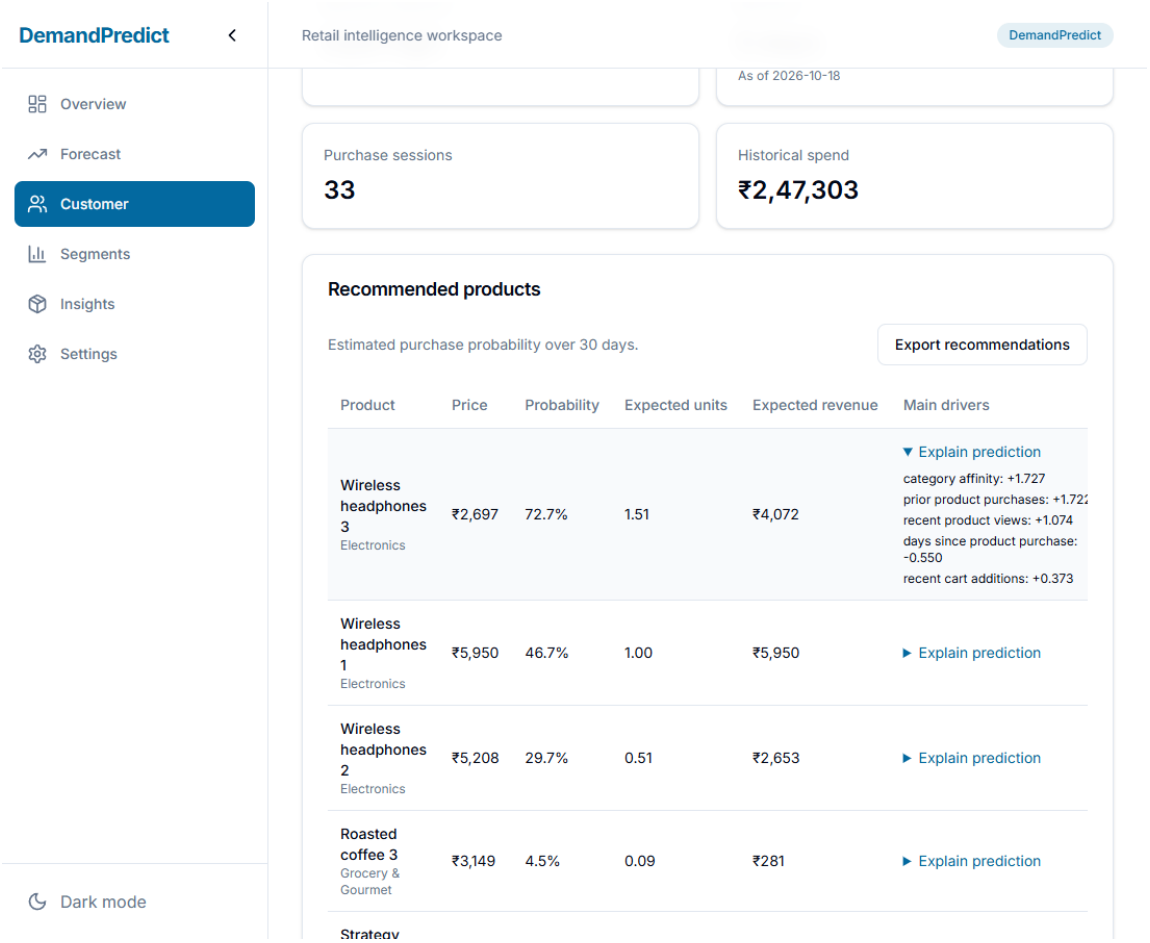


Figure 5.3 Customer recommendations and explanation

This viewport excerpt shows a selected customer's recommended products, purchase probabilities, expected quantities and expected revenue. Expanding a prediction displays the strongest positive and negative linear model contributions. Customer search supports name, email and customer ID.

The explanation describes the 30-day logistic-regression score in log-odds units. It identifies associations used by the fitted model, such as prior product activity and category affinity. It does not prove that changing a feature would cause a purchase. The full customer page also includes recent activity.

5.2 Application Outputs (continued)

Data import

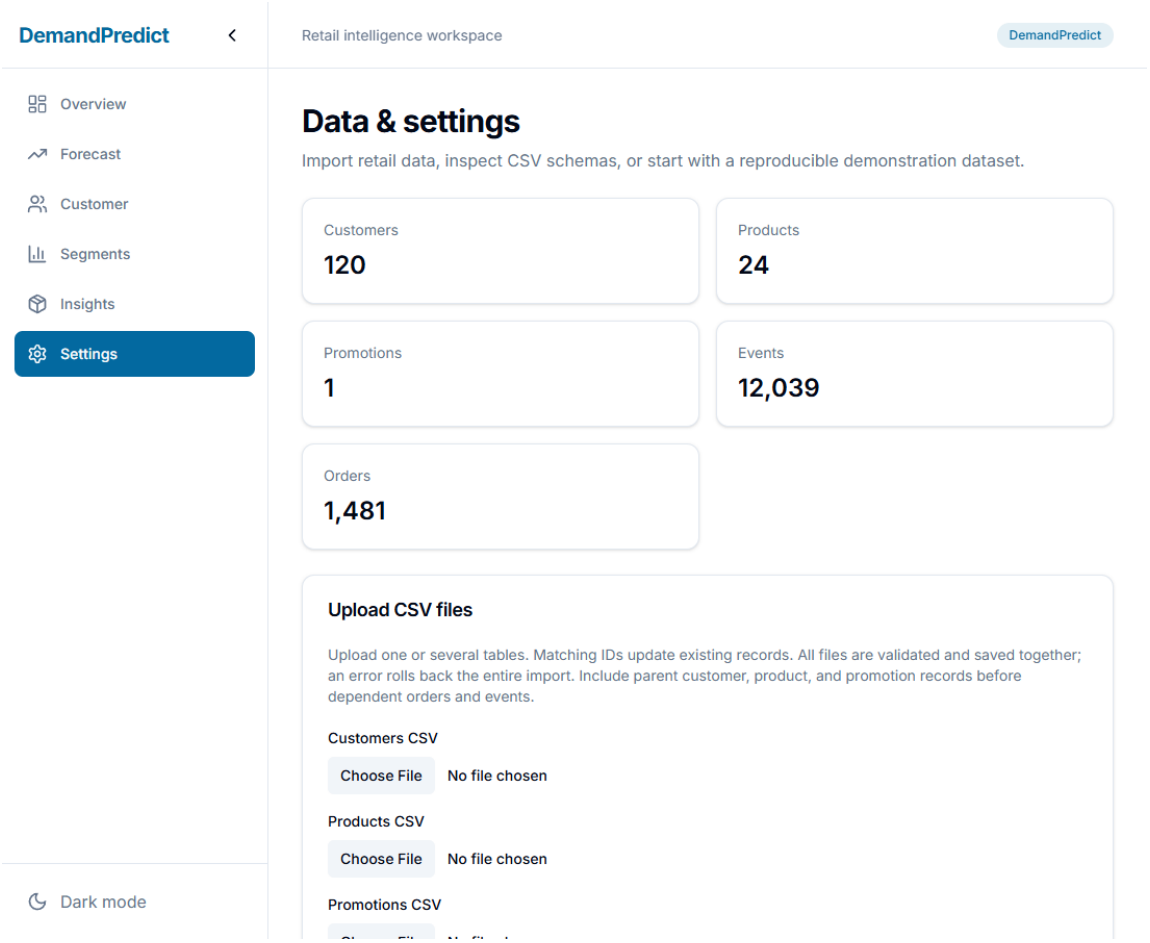


Figure 5.4 Dataset import and status interface

The Settings screen displays database counts and provides upload controls for customers, products, promotions, events and orders. Schema examples and downloadable CSV templates guide preparation. The screenshot shows the existing demonstration data and the beginning of the upload form.

Validation failures return row-specific feedback. An invalid file does not partially update the other supplied files. The demo action is restricted to an empty database, and repeated IDs update existing records instead of adding duplicates. The workflow keeps data preparation visible to the user.

5.2 Application Outputs (continued)

Customer segments

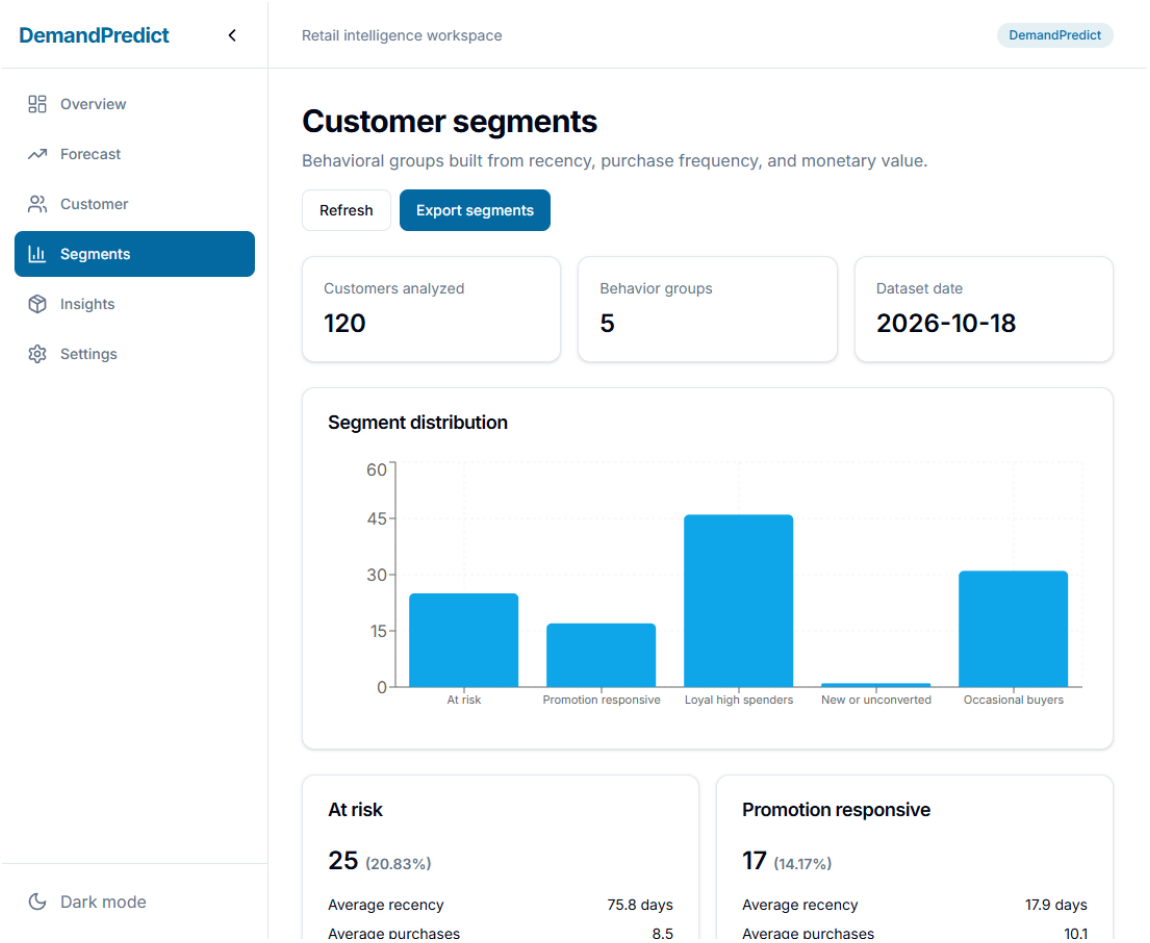


Figure 5.5 Customer segmentation interface

The Segments screen displays the distribution of behaviour-based groups and their summary RFM measures. In the captured demonstration, 120 customers are assigned to five labelled groups, including 25 at-risk customers. Cards provide recommendations and access to paginated customer lists.

Labels summarize observed activity and rule-based overrides. They are not validated predictions of future churn or guaranteed responses to a promotion. The interface is intended to help users identify groups for further investigation.

5.2 Application Outputs (continued)

Inventory insights

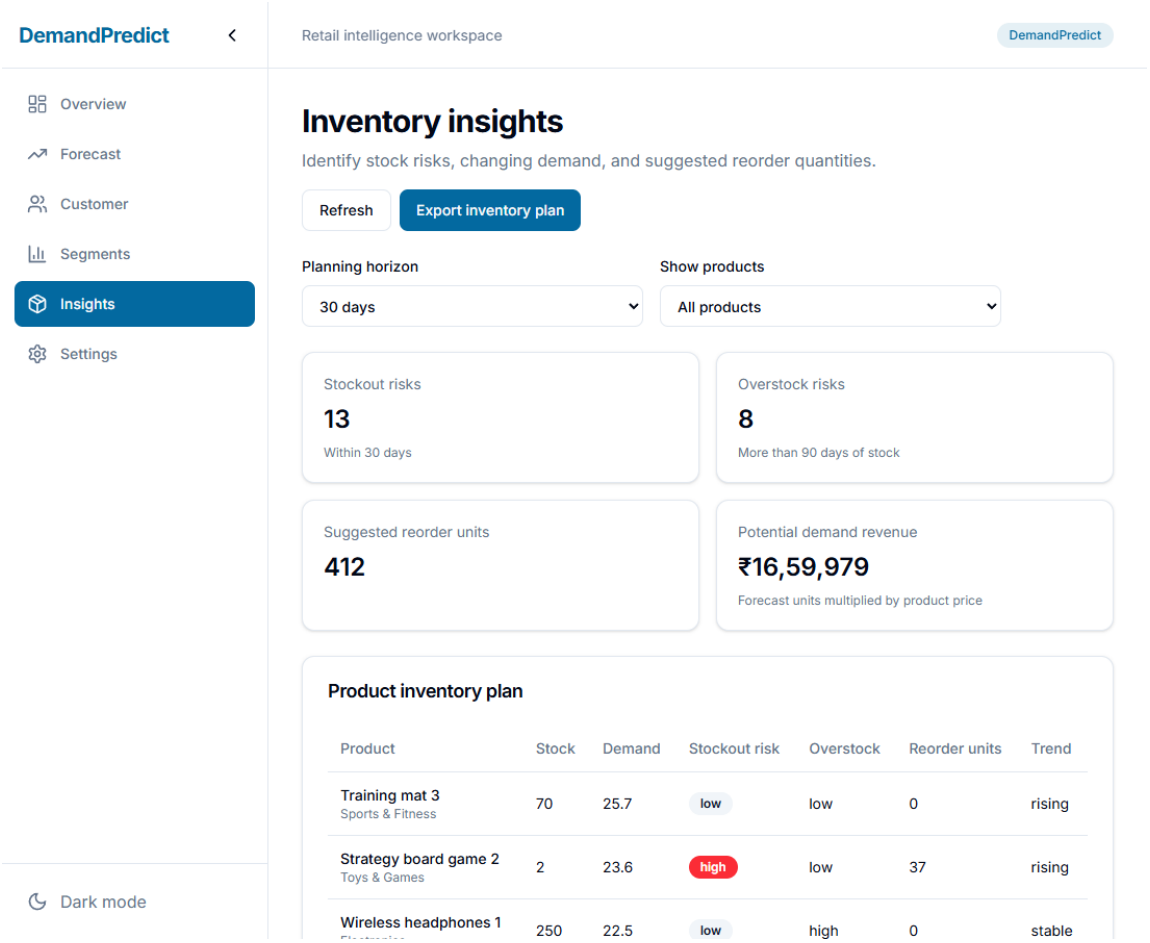


Figure 5.6 Inventory planning interface

The Insights screen combines stock, recent daily demand and the selected planning horizon. The captured 30-day view identifies 13 stockout risks, eight overstock risks and 412 suggested reorder units. The product table exposes stock, expected demand, risk and demand trend for review.

These outputs are deterministic planning estimates from the snapshot. Potential demand revenue multiplies projected units by price; it is not guaranteed future revenue. Reorder suggestions exclude supplier lead times, inbound orders and historical stockout censoring, and never execute a purchase order.

5.3 Testing and Verification

Verification combined backend tests with an end-to-end browser workflow. The backend suite checked API behaviour, data import integrity and model edge cases. Browser checks exercised user-facing actions on the demonstration database rather than relying only on isolated component tests.

Table 5.3 Verification summary

Area	Checks and observed outcome
Backend suite	32 pytest tests passed across API, import and model tests.
Import integrity	Invalid schemas and references rejected; transactional behaviour, duplicate IDs and unsafe values covered.
Browser workflow	One comprehensive Playwright workflow passed across six screens, downloads, import rejection and responsive navigation.
Presentation	Desktop screenshots and a mobile dark-theme screenshot captured from the running application.
Static checks	TypeScript/production build, ESLint, formatting and Ruff checks passed during project completion.
Dependency checks	npm and Python dependency audits reported no findings at the time of verification.
Container configuration	Dockerfiles and compose configuration are included; container execution was not verified because Docker was unavailable.

Inference from verification

The checks support that the implemented workflow runs on the local demonstration environment and handles the tested invalid inputs. They do not establish unlimited dataset capacity, security against all attacks, or production reliability. No throughput, latency or large-scale load benchmark was performed.

Evaluation data, model settings and screenshots are retained with the project. The saved JSON provides machine-readable forecast and purchase-model metrics. Re-running the generator with a different end date or importing different records will change the outputs, so comparisons must record the dataset snapshot as well as the random seed.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

DemandPredict provides a complete local workflow for importing retail records and analysing demand, customer purchase propensity, behavioural segments and inventory needs. React and FastAPI connect the analytical modules to an interactive interface, while SQLite and validated transactional imports maintain consistent source data.

The project implements several forecasting approaches with chronological validation rather than presenting a single unsupported accuracy claim. Results on the demonstration dataset show that the best forecast method depends on the category. The logistic-regression customer model improves Brier score over a constant-probability baseline on its validation window and provides interpretable linear feature contributions.

The application also demonstrates practical engineering requirements: input validation, dataset-aware model caching, error and empty states, responsive navigation, CSV export and automated verification. These features make the analytical workflow repeatable and accessible for project demonstration and further development.

The evidence remains limited to synthetic data and a local environment. Approximate forecast bands, constant-hazard probability conversion and inventory rules rely on simplifying assumptions. The system supports informed human review; it does not establish business gains or replace operational judgement.

6.2 Future Scope

Future evaluation should use authorized real retail history, multiple chronological backtests and category-specific baselines for intermittent demand. Calibration checks and measured forecast-interval coverage would improve interpretation. Promotion, price and external calendar effects could be added where reliable historical features are available.

Inventory planning can be extended with supplier lead times, service-level objectives, inbound purchase orders and stockout-censored demand. Customer models can be monitored for drift and assessed using outcomes from appropriately designed business experiments.

For broader deployment, the application requires authentication, authorization, HTTPS, managed secrets, backups and privacy controls. Background model jobs, scalable storage and caching would support larger datasets. Container execution and deployment tests should be completed before claiming operational readiness.

REFERENCES

[1] S. J. Taylor and B. Letham, *Forecasting at Scale*, PeerJ Preprints, 2017. DOI: 10.7287/peerj.preprints.3190v2. https://peerj.com/preprints/3190/

[2] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016. DOI: 10.1145/2939672.2939785. https://arxiv.org/abs/1603.02754

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*, Advances in Neural Information Processing Systems 30, 2017. NeurIPS 2017 proceedings (online).

[4] S. M. Lundberg and S.-I. Lee, *A Unified Approach to Interpreting Model Predictions*, Advances in Neural Information Processing Systems 30, 2017. https://arxiv.org/abs/1705.07874

[5] DemandPredict project repository, *Implementation and operating instructions*, local source snapshot, October 2026. Key files: README.md; ml/forecasting.py; ml/recommendations.py; backend/app/services/analytics.py; backend/app/services/upload_service.py.

[6] DemandPredict project repository, *Saved model evaluation*, models/evaluation.json, data as of 18 October 2026. Contains category forecast metrics, validation settings and purchase-propensity results.

[7] DemandPredict project repository, *Verification sources and application captures*, tests/test_api.py; tests/test_imports.py; tests/test_models.py; frontend/e2e/workflows.spec.ts; frontend/test-results. Screenshots used in this report were captured from the running demonstration application.

Online research references accessed 19 October 2026. Project results refer to the saved demonstration-data snapshot and the local implementation described in this report.